

MANUEL D'APPRENTISSAGE PAR LA PRATIQUE

SPRINT 3 — TÂCHE 02 : AFFECTATION DES COLLABORATEURS AU PROJET

Guide d'ingénierie active pour programmer manuellement l'API complète d'affectation d'équipe de chantier avec les rôles Conducteur de travaux, Chef de Chantier et Consultant lecture (US-04).

Sprint : Sprint 3 (Gestion de Projets & RBAC)	Périmètre : apps/projets & apps/core
Rôle Développeur : Lead Backend Django / DRF	Niveau de Compétence Cible : <i>Homo Docens</i>
Architecture : Multi-Tenant PostgreSQL & RBAC	Spécification : US-04 [P0] Contrôle d'accès granulaire

Cadrage Stratégique SCQA (Minto Pyramid)

Situation : Dans CCD Digital, un chantier BTP est opéré par une équipe pluridisciplinaire rattachée au projet via l'entité AffectationProjet.

Complication : Si les affectations initiales sont créées lors du wizard de création du projet, il n'existe pas encore d'API dédiée et modulaire permettant au Chef de Projet ou à la Direction de gérer dynamiquement les affectations tout au long de la vie du chantier. De plus, il est crucial d'assigner rigoureusement les 3 rôles clés de terrain : **Conducteur de travaux** (gestion opérationnelle et validation), **Chef de Chantier** (saisie terrain et pointages) et **Consultant lecture** (consultation sans droit d'écriture), tout en protégeant l'invariant vital US-04 : *chaque projet doit avoir en permanence au moins 1 Chef de Projet actif*.

Question : Comment concevoir des endpoints RESTful imbriqués `/api/v1/projets/{id}/affectations/`, une couche de service robuste, des validations strictes et des tests automatisés complets ?

Orientation : Ce manuel décompose chaque brique technique pour que vous tapiez chaque ligne de code de vos propres mains.

Neuro-Pédagogie Active : Zéro Copier-Coller

Le Principe du Recyclage Neuronale & de l'Automatisation Moteur (Stanislas Dehaene) :

Ne déléguiez pas l'écriture du code à un générateur. La compréhension passive est une illusion de compétence (Kahneman). En tapant manuellement chaque instruction de sérialiseur, de vue et de service, vos circuits neuronaux de prédiction s'ajustent (boucle bayésienne), fixant les automatismes de l'ORM Django jusqu'à l'expertise réflexe.

SECTION 1 : FILM MENTAL & OBJECTIF SACRÉ (JOSEPH MURPHY)

1.1 La Visualisation de la Réussite Finale (Le Film Mental)

Installez-vous confortablement. Fermez les yeux pendant soixante secondes. Visualisez le résultat parfait de votre travail : Le Chef de Projet se connecte à l'application web. Il consulte la fiche du chantier de 'Résidence Ivoire Prestige'. Il ouvre l'onglet **Équipe du Chantier**. L'API GET /api/v1/projets/{id}/affectations/ répond en 25 ms, renvoyant la liste ordonnée des membres avec leurs avatars, téléphones, rôles lisibles et dates. Il clique sur 'Affecter un collaborateur', choisit Moussa Diallo avec le rôle **Conducteur de travaux**, puis Fatou Koné avec le rôle **Consultant lecture**. La requête POST crée les affectations instantanément. Puis, s'il essaie par inadvertance de révoquer l'unique Chef de Projet, l'API intercepte l'action et renvoie un refus clair et protecteur (HTTP 400). Les 8 tests pytest passent tous au vert. Vous ressentez le calme et la fierté d'un travail d'ingénieur accompli avec rigueur.

Auto-Suggestion & Conditionnement Mental

Affirmation d'Ancrage Subconscient :

« Mon subconscient est une source inépuisable de précision et d'ordre. Je maîtrise l'architecture des autorisations et la relation entre Projet et Utilisateur. Mon code protège les invariants du métier BTP avec une solidité inébranlable. Chaque ligne que je saisis renforce ma certitude et mon statut d'ingénieur souverain. »

1.2 La Loi de l'Effort Inversé & Gestion de l'Erreur (Murphy & Dehaene)

Lorsque le cerveau rencontre une erreur (par exemple un code HTTP 403 inattendu ou un échec d'unicité), la réaction réflexe est souvent l'agacement ou la précipitation. **Appliquez la Loi de l'Effort Inversé** : plus vous luttez sous la tension, moins votre esprit conscient a accès aux solutions créatives de votre subconscient. Respirez lentement. Considérez l'erreur non pas comme une faute, mais comme un *signal d'ajustement bayésien* (Dehaene, Pilier 3). Le test qui échoue vous indique exactement où se situe l'écart entre la réalité du code et le modèle mental.

1.3 Les 4 Objectifs Sacrés de la Tâche

Objectif	Composant Clé	Critère de Réussite Inviolable
1. Rôles Métier BTP	apps/core/enums.py RoleProjet	Support explicite de CONDUCTEUR_TRAVAUX ('CT'), CHEF_CHANTIER ('CC') et VISITEUR ('VI' avec libellé 'Consultant lecture').
2. Lister l'Équipe Chantier	GET /projets/{id}/affectations/	Renvoie tous les collaborateurs affectés au projet avec leurs rôles, dates et statut actif. Requête sans N+1 (select_related).
3. Affecter un Collaborateur	POST /projets/{id}/affectations/	Création ou réactivation propre. Contrôle de l'appartenance au même tenant. Unicité par utilisateur et par projet.
4. Invariant Chef de Projet	PATCH/DELETE Contrôle d'intégrité US-04	Interdiction formelle de désactiver ou supprimer l'affectation du dernier Chef de Projet actif sur le chantier.

SECTION 2 : ARCHITECTURE & INVARIANTS (PÓLYA / DEHAENE)

2.1 Les Invariants Inviolables de George Pólya

Avant de concevoir l'algorithme, George Pólya prescrit d'isoler les contraintes qui ne doivent être violées sous aucun prétexte. Pour notre système d'affectation :

- **Invariant 1 (Gouvernance US-04 : 1 CP Actif Minimum)** : Un chantier ne peut jamais se retrouver 'orphelin'. Il doit comporter en permanence au moins **un Chef de Projet actif**. Toute tentative de désactivation (`est_actif=False`) ou de suppression de l'unique CP actif est rejetée avec une erreur 400 Bad Request.
- **Invariant 2 (Habilitation d'Affectation RBAC)** : Seuls les administrateurs du tenant (DG, ADMIN, Propriétaire) OU le Chef de Projet assigné au chantier (`projet.chef_projet_id == user.id`) ont le privilège d'ajouter, modifier ou révoquer une affectation. Les autres membres reçoivent un 403 Forbidden.
- **Invariant 3 (Unicité de l'Association)** : Un collaborateur ne peut avoir qu'une seule affectation sur un chantier donné (contrainte SQL `uq_affectation_utilisateur_projet`). Si une affectation désactivée existait déjà dans le passé, l'API la réactive avec le nouveau rôle et les nouvelles dates plutôt que d'échouer.
- **Invariant 4 (Étanchéité Multi-Tenant)** : Le collaborateur affecté doit obligatoirement exister dans le schéma du tenant courant. Impossible d'affecter un utilisateur d'une autre entreprise cliente.
- **Invariant 5 (Matrice des Rôles Spécifiés)** : Les rôles autorisés lors de l'affectation couvrent notamment : `CONDUCTEUR_TRAVAUX` ('CT'), `CHEF_CHANTIER` ('CC') et `VISITEUR` ('VI', affiché 'Consultant lecture').

2.2 Schéma Conceptuel de Données & Flux RESTful

Fichier : Structure Relationnelle AffectationProjet

Projet		AffectationProjet		Utilisateur
id: UUID (PK)	1	id: UUID (PK)	0..*	id: UUID (PK)
reference: CharField	<----->	projet: FK(Projet)	<----->	email: EmailField
nom: CharField	0..*	utilisateur: FK(Utilisateur)	1	nom, prenom: CharField
chef_projet: FK(User)		role_projet: CharField(5)		role_global: CharField
conducteur_travaux: FK		(CT, CC, VI / Consultant lec)		role_personnalise: FK
statut: CharField		date_debut, date_fin: Date		telephone: CharField
		est_actif: BooleanField		statut: ACTIF / INVITE
		[uq_utilisateur_projet]		

2.3 Contrat d'Interface des Endpoints RESTful

Méthode & URL	Rôle & Action Métier	Codes HTTP Attendus
GET /api/v1/projets/{projet_id}/affectations/	Lister tous les intervenants affectés au chantier (actifs et inactifs avec filtre <code>?actifs_seulement=true</code>).	200 OK 401 Unauthorized 404 Not Found
POST /api/v1/projets/{projet_id}/affectations/	Affecter un collaborateur au chantier avec son rôle (CT, CC, VI) et ses dates d'intervention.	201 Created 400 Bad Request 403 Forbidden
PATCH /api/v1/projets/{projet_id}/affectations/{id}/	Modifier le rôle, prolonger/ajuster les dates ou réactiver/désactiver le collaborateur.	200 OK 400 Bad Request (Invariant CP) 403 Forbidden
DELETE /api/v1/projets/{projet_id}/affectations/{id}/	Supprimer physiquement une affectation erronée (avec vérification stricte de l'invariant CP).	204 No Content 400 Bad Request (Dernier CP) 403 Forbidden

SECTION 3 : GUIDE D'IMPLÉMENTATION PAS-À-PAS POUR VOS MAINS

Consigne d'Or : Tapez chaque instruction manuellement dans votre éditeur (VS Code / Cursor). Ne faites aucun copier-coller. Observez l'autocomplétion, comprenez les imports et la logique métier.

3.1 Étape 1 : Mettre à jour l'Énumération des Rôles de Projet

Ouvrez le fichier `apps/core/enums.py`. Dans la classe `RoleProjet`, mettez à jour le libellé du rôle `VISITEUR` pour correspondre à **'Consultant lecture'** et déclarez un alias explicite `CONSULTANT_LECTURE = VISITEUR` :

Fichier : `apps/core/enums.py`

```
class RoleProjet(models.TextChoices):
    DIRECTEUR_PROJET = "DP", "Directeur de Projet"
    CHEF_PROJET = "CP", "Chef de Projet"
    CONDUCTEUR_TRAVAUX = "CT", "Conducteur de Travaux"
    CHEF_CHANTIER = "CC", "Chef de Chantier"
    INGENIEUR_TECHNICIEN = "IT", "Ingénieur / Technicien"
    DIRECTEUR_FINANCIER = "DF", "Directeur Financier"
    MAGASINIER = "MAG", "Magasinier"
    SOUS_TRAITANT = "ST", "Sous-traitant"
    FOURNISSEUR = "FRN", "Fournisseur"
    MAITRE_OUVRAGE = "MOA", "Maître d'Ouvrage (Client)"
    VISITEUR = "VI", "Consultant lecture" # Rôle de consultation en lecture seule (US-04)

# Alias sémantique conforme à la spécification US-04
RoleProjet.CONCONSULTANT_LECTURE = RoleProjet.VISITEUR
```

3.2 Étape 2 : Créer la Couche Service d'Affectation

Créez ou ouvrez le fichier `apps/projets/services/affectations.py`. Vous allez y coder les fonctions métier atomiques pour lister, créer, mettre à jour et révoquer une affectation :

Fichier : apps/projets/services/affectations.py (Partie 1)

```

"""Services métier pour la gestion des affectations d'équipe au chantier (US-04)."""

from django.core.exceptions import ValidationError
from django.db import transaction
from django.utils import timezone
from django.utils.translation import gettext_lazy as _

from apps.accounts.models import Utilisateur
from apps.core.enums import RoleProjet
from apps.projets.models import AffectationProjet, Projet

def lister_affectations_projet(projet: Projet, actifs_seulement: bool = False):
    """Renvoie les affectations du projet avec chargement optimisé de l'utilisateur."""
    qs = (
        AffectationProjet.objects.filter(projet=projet, supprime_le__isnull=True)
        .select_related("utilisateur", "role")
        .order_by("role_projet", "utilisateur__nom")
    )
    if actifs_seulement:
        qs = qs.filter(est_actif=True)
    return qs

def affecter_collaborateur_projet(
    *,
    projet: Projet,
    utilisateur: Utilisateur,
    role_projet: str,
    date_debut=None,
    date_fin=None,
    role_personnalise=None,
    modifie_par: Utilisateur | None = None,
) -> AffectationProjet:
    """Affecte ou réactive un collaborateur sur un chantier avec un rôle spécifique."""
    if role_projet not in RoleProjet.values:
        raise ValidationError(_("Le rôle de projet spécifié n'est pas valide. "))

    if date_debut and date_fin and date_fin < date_debut:
        raise ValidationError(_("La date de fin ne peut pas être antérieure à la date de début. "))

    date_debut_effective = date_debut or timezone.now().date()

    with transaction.atomic():
        affectation, cree = AffectationProjet.objects.get_or_create(
            projet=projet,
            utilisateur=utilisateur,
            defaults={
                "role_projet": role_projet,
                "role": role_personnalise,
                "date_debut": date_debut_effective,
                "date_fin": date_fin,
                "est_actif": True,
                "cree_par": modifie_par,
            },
        )
    if not cree:
        # Si l'affectation existait déjà (éventuellement inactive), on la réactive et met à jour le rôle
        affectation.role_projet = role_projet
        if role_personnalise is not None:
            affectation.role = role_personnalise
        affectation.date_debut = date_debut_effective
        affectation.date_fin = date_fin
        affectation.est_actif = True
        affectation.modifie_par = modifie_par
        affectation.save(update_fields=["role_projet", "role", "date_debut", "date_fin", "est_actif", "modifie_par", "modifie"])

    # Synchronisation du rôle principal sur le Projet si conducteur de travaux
    if role_projet == RoleProjet.CONDUCTEUR_TRAVAUX and projet.conducteur_travaux_id != utilisateur.id:
        projet.conducteur_travaux = utilisateur
        projet.save(update_fields=["conducteur_travaux", "modifie_le"])

    return affectation

```

3.2 Couche Service (Suite) : Modification & Révocation avec Invariant CP

Ajoutez dans `apps/projets/services/affectations.py` les fonctions de mise à jour et de révocation. Notez le respect intransigeant de l'**Invariant US-04** :

Fichier : `apps/projets/services/affectations.py` (Partie 2)

```
def verifier_invariant_chef_projet(projet: Projet, affectation_a_exclure_id=None):
    """Vérifie qu'il reste au moins un Chef de Projet actif sur le chantier (US-04)."""
    qs = AffectationProjet.objects.filter(
        projet=projet,
        role_projet=RoleProjet.CHEF_PROJET,
        est_actif=True,
        supprime_le__isnull=True,
    )
    if affectation_a_exclure_id:
        qs = qs.exclude(id=affectation_a_exclure_id)
    if not qs.exists():
        raise ValidationError(_("Impossible de désactiver ou supprimer le dernier Chef de Projet actif du chantier."))

def modifier_affectation_projet(*, affectation: AffectationProjet, donnees: dict, modifie_par: Utilisateur | None = None) -> AffectationProjet:
    """Met à jour le rôle ou les dates d'une affectation avec protection de l'invariant CP."""
    nouveau_statut_actif = donnees.get("est_actif")
    nouveau_role = donnees.get("role_projet")

    # Si on désactive ou si on change le rôle d'un Chef de Projet, vérifier qu'un autre CP reste actif
    if affectation.role_projet == RoleProjet.CHEF_PROJET:
        if nouveau_statut_actif is False or (nouveau_role and nouveau_role != RoleProjet.CHEF_PROJET):
            verifier_invariant_chef_projet(affectation.projet, affectation_a_exclure_id=affectation.id)

    champs_maj = ["modifie_le"]
    for champ in ["role_projet", "date_debut", "date_fin", "est_actif"]:
        if champ in donnees:
            setattr(affectation, champ, donnees[champ])
            champs_maj.append(champ)

    if "role" in donnees:
        affectation.role = donnees["role"]
        champs_maj.append("role")

    affectation.modifie_par = modifie_par
    champs_maj.append("modifie_par")
    affectation.save(update_fields=champs_maj)
    return affectation

def revoquer_affectation_projet(*, affectation: AffectationProjet, suppression_physique: bool = False, modifie_par: Utilisateur | None = None):
    """Révoque ou supprime une affectation après validation de l'invariant US-04."""
    if affectation.role_projet == RoleProjet.CHEF_PROJET and affectation.est_actif:
        verifier_invariant_chef_projet(affectation.projet, affectation_a_exclure_id=affectation.id)

    if suppression_physique:
        affectation.delete()
    else:
        affectation.est_actif = False
        affectation.modifie_par = modifie_par
        affectation.save(update_fields=["est_actif", "modifie_par", "modifie_le"])
```

3.3 Étape 3 : Créer les Sérialiseurs DRF pour les Affectations

Créez le fichier `apps/projets/serializers/affectation.py` pour définir les contrats d'entrée et de sortie :

Fichier : apps/projets/serializers/affectation.py

```
"""Sérialiseurs pour l'API d'affectation des collaborateurs aux projets (US-04)."""

from rest_framework import serializers
from apps.accounts.models import Role, Utilisateur
from apps.core.enums import RoleProjet
from apps.projets.models import AffectationProjet

class IntervenantProjetDetailSerializer(serializers.Serializer):
    id = serializers.UUIDField()
    nom = serializers.CharField()
    prenom = serializers.CharField(allow_blank=True, default="")
    nom_complet = serializers.CharField()
    email = serializers.EmailField()
    telephone = serializers.CharField(allow_blank=True, default="")
    statut = serializers.CharField()

class AffectationProjetResponseSerializer(serializers.ModelSerializer):
    utilisateur = IntervenantProjetDetailSerializer(read_only=True)
    role_projet_libelle = serializers.CharField(source="get_role_projet_display", read_only=True)
    role_personnalise_code = serializers.CharField(source="role.code", read_only=True, default=None)

class Meta:
    model = AffectationProjet
    fields = [
        "id",
        "projet_id",
        "utilisateur",
        "role_projet",
        "role_projet_libelle",
        "role_personnalise_code",
        "date_debut",
        "date_fin",
        "est_actif",
        "cree_le",
    ]

class AffectationProjetCreateSerializer(serializers.Serializer):
    utilisateur_id = serializers.UUIDField(required=True)
    role_projet = serializers.ChoiceField(choices=RoleProjet.choices, required=True)
    date_debut = serializers.DateField(required=False, allow_null=True)
    date_fin = serializers.DateField(required=False, allow_null=True)
    role_personnalise_id = serializers.UUIDField(required=False, allow_null=True)

    def validate(self, attrs):
        user_id = attrs.get("utilisateur_id")
        user = Utilisateur.objects.filter(id=user_id, supprime_le__isnull=True).first()
        if not user:
            raise serializers.ValidationError({"utilisateur_id": "Collaborateur introuvable dans cette entreprise."})
        attrs["utilisateur_instance"] = user

        role_id = attrs.get("role_personnalise_id")
        if role_id:
            role = Role.objects.filter(id=role_id, supprime_le__isnull=True).first()
            if not role:
                raise serializers.ValidationError({"role_personnalise_id": "Rôle personnalisé introuvable."})
            attrs["role_instance"] = role
        return attrs
```

3.4 Étape 4 : Développer les Vues DRF d'Affectation

Créez le fichier `apps/projets/views/affectation.py`. Vous y programmez les contrôles d'habilitation RBAC et la gestion des réponses HTTP :

Fichier : `apps/projets/views/affectation.py` (Partie 1)

```
"""Vues API pour la gestion de l'équipe de chantier / affectations (US-04)."""

from django.core.exceptions import ValidationError as DjangoValidationError
from django.shortcuts import get_object_or_404
from rest_framework import status
from rest_framework.exceptions import PermissionDenied, ValidationError
from rest_framework.parsers import JSONParser
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework.views import APIView

from apps.core.enums import RoleGlobal
from apps.projets.models import AffectationProjet, Projet
from apps.projets.serializers.affectation import (
    AffectationProjetCreateSerializer,
    AffectationProjetResponseSerializer,
)
from apps.projets.services.affectations import (
    affecter_collaborateur_projet,
    lister_affectations_projet,
    modifier_affectation_projet,
    revoquer_affectation_projet,
)

def _verifier_droits_gestion_equipe(user, projet: Projet):
    """Autorise l'administrateur, le DG ou le Chef de Projet assigné à ce chantier."""
    est_admin = (
        getattr(user, "is_owner", False)
        or getattr(user, "is_dg", False)
        or getattr(user, "role_global", None) in (RoleGlobal.ADMIN, RoleGlobal.DIRECTEUR_GENERAL)
        or getattr(user, "is_superuser", False)
    )
    est_cp_du_projet = (projet.chef_projet_id == user.id or user.role_global == RoleGlobal.CHEF_PROJET)
    if not (est_admin or est_cp_du_projet):
        raise PermissionDenied("Seul le Chef de Projet assigné ou la Direction peut gérer l'équipe du chantier.")

class ProjetAffectationListCreateView(APIView):
    """GET et POST /api/v1/projets/{projet_id}/affectations/."""
    permission_classes = [IsAuthenticated]
    parser_classes = [JSONParser]

    def get(self, request, projet_id):
        projet = get_object_or_404(Projet, pk=projet_id, supprime_le__isnull=True)
        actifs_seulement = request.query_params.get("actifs_seulement", "false").lower() == "true"
        qs = lister_affectations_projet(projet, actifs_seulement=actifs_seulement)
        serializer = AffectationProjetResponseSerializer(qs, many=True)
        return Response(serializer.data, status=status.HTTP_200_OK)

    def post(self, request, projet_id):
        projet = get_object_or_404(Projet, pk=projet_id, supprime_le__isnull=True)
        _verifier_droits_gestion_equipe(request.user, projet)
        serializer = AffectationProjetCreateSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        try:
            aff = affecter_collaborateur_projet(
                projet=projet,
                utilisateur=serializer.validated_data["utilisateur_instance"],
                role_projet=serializer.validated_data["role_projet"],
                date_debut=serializer.validated_data.get("date_debut"),
                date_fin=serializer.validated_data.get("date_fin"),
                role_personnalise=serializer.validated_data.get("role_instance"),
                modifie_par=request.user,
            )
        except DjangoValidationError as exc:
            raise ValidationError({"detail": str(exc.message if hasattr(exc, 'message') else exc)})
        return Response(AffectationProjetResponseSerializer(aff).data, status=status.HTTP_201_CREATED)
```


3.5 Étape 5 : Déclarer les Routes URL

Ouvrez `apps/projets/urls.py` et ajoutez les deux routes RESTful imbriquées pour les affectations :

Fichier : `apps/projets/urls.py`

```
from apps.projets.views.affectation import (
    ProjetAffectationDetailView,
    ProjetAffectationListCreateView,
)

urlpatterns = [
    # ... routes existantes ...
    path("projets/<uuid:projet_id>/affectations/", ProjetAffectationListCreateView.as_view(), name="projet-affectations-liste"),
    path("projets/<uuid:projet_id>/affectations/<uuid:pk>/", ProjetAffectationDetailView.as_view(), name="projet-affectation-detail")
]
```

SECTION 4 : SIGNAL D'ERREUR BAYÉSIEN & PRE-MORTEM (KAHNEMAN)

Dans la technique du **Pre-Mortem** (Gary Klein & Daniel Kahneman), nous nous projetons dans le scénario où la fonctionnalité est livrée et subit des dysfonctionnements critiques. Voici l'analyse préventive :

Scénario de Panne Potentiel	Mécanisme de Défaillance	Parade Préventive Programmée
1. Chantier sans Chef de Projet (Orphelin)	Un utilisateur désactive ou supprime l'affectation du CP, laissant le chantier sans responsable habilité.	La fonction <code>verifier_invariant_chef_projet</code> intercepte l'action et lève une exception HTTP 400 si aucun autre CP n'est actif.
2. Collision de Clé Unique SQL	Une affectation inactive existait dans le passé. Une nouvelle affectation échoue sur <code>uq_affectation_utilisateur_projet</code> .	Utilisation de <code>get_or_create</code> puis réactivation explicite avec mise à jour du rôle et des dates.
3. Usurpation par Ouvrier / Tiers	Un chef de chantier ou un sous-traitant s'auto-attribue des droits de Chef de Projet via l'API.	Contrôle d'autorisation strict dans <code>_verifier_droits_gestion_equipe</code> (seul DG, Admin ou CP peut modifier l'équipe).
4. Explosion N+1 Queries SQL	La liste de 30 affectations déclenche 30 requêtes SQL pour charger les profils et rôles.	Utilisation systématique de <code>select_related('utilisateur', 'role')</code> garantissant 1 seule requête SQL.
5. Dates Incohérentes	La date de fin est antérieure à la date de début de mission sur le chantier.	Validation dans le service : <code>date_fin >= date_debut</code> obligatoire.

SECTION 5 : CHECKLIST DE TESTS AUTOMATISÉS PYTEST

Pour ancrer vos apprentissages (Pilier 2 et 3 de Dehaene : Engagement Actif & Retour sur Erreur), vous allez taper manuellement le fichier de test `apps/projets/tests/test_affectations_equipe.py` :

Fichier : `apps/projets/tests/test_affectations_equipe.py`

```
import pytest
from rest_framework import status
from apps.core.enums import RoleProjet
from apps.projets.models import AffectationProjet

@pytest.mark.django_db
def test_affecter_conducteur_travaux_et_consultant_lecture(client_tenant, admin_user, projet_db, collaborateur_user):
    """Vérifie l'affectation réussie avec CT et Consultant lecture (VI)."""
    client = _auth(client_tenant, admin_user)
    url = f"/api/v1/projets/{projet_db.id}/affectations/"

    # 1. Affectation Conducteur de Travaux
    payload_ct = {"utilisateur_id": str(collaborateur_user.id), "role_projet": RoleProjet.CONDUCTEUR_TRAVAUX}
    rep_ct = client.post(url, payload_ct, format="json")
    assert rep_ct.status_code == status.HTTP_201_CREATED
    assert rep_ct.json()["role_projet"] == RoleProjet.CONDUCTEUR_TRAVAUX

    # 2. Mise à jour vers Consultant lecture (VISITEUR)
    aff_id = rep_ct.json()["id"]
    rep_patch = client.patch(f"{url}{aff_id}/", {"role_projet": RoleProjet.VISITEUR}, format="json")
    assert rep_patch.status_code == status.HTTP_200_OK
    assert rep_patch.json()["role_projet_libelle"] == "Consultant lecture"

@pytest.mark.django_db
def test_protection_invariant_dernier_chef_projet(client_tenant, admin_user, projet_db, cp_user):
    """Empêche formellement de désactiver ou supprimer le dernier Chef de Projet (US-04)."""
    client = _auth(client_tenant, admin_user)
    aff_cp = AffectationProjet.objects.get(projet=projet_db, utilisateur=cp_user)
    url_detail = f"/api/v1/projets/{projet_db.id}/affectations/{aff_cp.id}/"

    # Tentative de désactivation
    rep_desact = client.patch(url_detail, {"est_actif": False}, format="json")
    assert rep_desact.status_code == status.HTTP_400_BAD_REQUEST
    assert "dernier chef de projet" in str(rep_desact.json()).lower()

    # Tentative de suppression
    rep_del = client.delete(url_detail)
    assert rep_del.status_code == status.HTTP_400_BAD_REQUEST
```

Protocole de Vérification en Terminal

Commandes d'exécution des tests dans le terminal :

```
.venv\Scripts\pytest.exe apps/projets/tests/test_affectations_equipe.py -v
```

Vérification de non-régression globale sur les projets :

```
.venv\Scripts\pytest.exe apps/projets/tests/test_creation_projet_v2.py -v
```

SECTION 6 : DÉFI HOMO DOCENS & CLÔTURE MÉTACOGNITIVE

6.1 Le Défi de l'Homo Docens (Enseigner pour Ancrer)

Dans la recherche cognitive (Principe 8 de Lovett et al.), l'apprenant devient un **Homo Docens** lorsqu'il est capable d'expliquer le 'pourquoi' de chaque arbitrage technique. Prenez une feuille ou votre carnet d'ingénieur et répondez à ces 3 questions sans regarder le code :

- **Question 1** : Pourquoi avoir choisi des endpoints imbriqués `/api/v1/projets/{id}/affectations/` plutôt qu'un endpoint plat `/api/v1/affectations/` ?
- **Question 2** : Comment la fonction `affecter_collaborateur_projet` gère-t-elle le cas où un collaborateur a déjà été affecté puis désactivé sur le chantier dans le passé sans violer la contrainte d'unicité SQL ?
- **Question 3** : Comment le système garantit-il l'invariant US-04 interdisant de laisser un chantier sans Chef de Projet actif ?

6.2 Rituel de Somnolence & Replay Neuronal Nocturne

Le sommeil lent et le sommeil paradoxal sont les phases où votre hippocampe réactive les assemblées neuronales sollicitées pendant l'exercice à un rythme accéléré ($20\times$), les intégrant dans les réseaux corticaux stables. Pour optimiser cette consolidation biologique :

Protocole de Consolidation Subconsciente

Rituel du Soir avant l'Endormissement :

1. **Déconnexion visuelle** : Éteignez vos écrans 30 minutes avant le coucher.
2. **Revue mentale** : Visualisez mentalement le cycle complet : la requête HTTP qui arrive, la validation DRF, la transaction atomique PostgreSQL et la mise à jour de l'équipe.
3. **Décharge subconsciente** : Dites-vous avec foi : « *Les compétences d'architecture et de sécurité BTP sont désormais gravées dans mon esprit. Demain, j'exécuterai chaque tâche avec aisance et souveraineté.* »

Validé par le Développeur :

Nom : _____
Date : ____ / ____ / 2026
Signature :

Approuvé par le Mentor Neurocognitif :

Agentic AI Pair Programmer & Cognitive Architect
Certification : CCD Digital Enterprise Grade
Statut : Conforme Sprint 3 LifeCycle